AI in a Box: Exploring Edge LLM Deployment, Benchmarking, and Mixed-Precision Compression



UNIVERSITY OF BIRMINGHAM

Portable Benchmarking and Recommendation

Part 1



UNIVERSITY OF BIRMINGHAM

Mixed-Precision GGUF Mixing Framework

Part 2



UNIVERSITY OF BIRMINGHAM

Motivation

UNIVERSITY OF BIRMINGHAM

- Running large language models(LLMs) on-device is attractive for **privacy**, **offline** availability, and **reduced dependence on cloud** infrastructure.
- But practical deployment is constrained by **limited RAM, compute**, **hardware variability.**

- **Quantisation** is the dominant practical mechanism for compressing large language models for local deployment: stored **weights** are **reduced** from **higher-precision** representations to **lower-bit formats such as Q4 or Q8**
- This **lowers model size** and memory use at the **cost of some quality**

- The “best” configuration depends on **memory usage**, **token throughput**, **prompt length**, **device**, and their specific **use-case**.
- A single quantisation choice is not universally best across CPU only, accelerated, and constrained devices.
- Static rules are also unreliable e.g. *smallest quantisation level that fits is not always best*

- My project looks at ways to make local LLM deployment more practical
  1. **data-driven**
  2. **automated**

Portable Benchmarking and Recommendation

Part 1



UNIVERSITY OF BIRMINGHAM

"Given a target device, a particular LLM model, and user-defined constraints such as memory budget, latency target, what configuration should I actually use?"

UNIVERSITY OF BIRMINGHAM

Run configuration

run_mtl0.yaml

configs > run_mtl0.yaml

```yaml
repo_id: Qwen/Qwen2.5-3B-Instruct-GGUF
model_dir: models/qwen2.5-3b

quants:
  - qwen2.5-3b-instruct-q3_k_m.gguf
  - qwen2.5-3b-instruct-q4_k_m.gguf
  - qwen2.5-3b-instruct-q6_k.gguf
  - qwen2.5-3b-instruct-q8_0.gguf

llama_bench_bin: llama-bench
time_bin: /usr/bin/time
time_args: ["-l"]

threads: 8
repetitions: 1
n_gen: 128

devices:
  - name: MTL0
    ngl: 80

prompt_tokens: [512, 1920, 3968, 8064]

raw_logs_dir: results/raw_logs
plots_dir: results/plots
runs_csv: results/runs.csv
recommendation_out: results/recommendation.json
```

Constraints configuration

edge_mtl0_8gb.yaml | recommendation.json

configs > edge_mtl0_8gb.yaml

```yaml
device: MTL0

n_prompt: 1920
n_gen: 128
min_n_prompt: 1920

max_rss_mb: 8192
min_headroom_mb: 750
max_ttft_s: 8.0
min_tg_tok_s: 10.0

topk: 5

prune:
  enabled: true
  timeout_s: 60
  stop_on_memory: true
  stop_on_predicted_ttft: true
  ttft_multiplier: 1.0
  apply_to_objectives:
    - balanced
    - speed_first
```



UNIVERSITY OF BIRMINGHAM

Framework Pipeline



- Orchestrates everything for you

UNIVERSITY OF BIRMINGHAM

Benchmark metrics logged into runs.csv

- Prompt-processing throughput, pp/s
- Generation throughput, tg/s
- TTFT proxy
- Peak RSS (RAM used by the process)
- Prompt scaling (prompt size)
- Repeatability means/stds

UNIVERSITY OF BIRMINGHAM

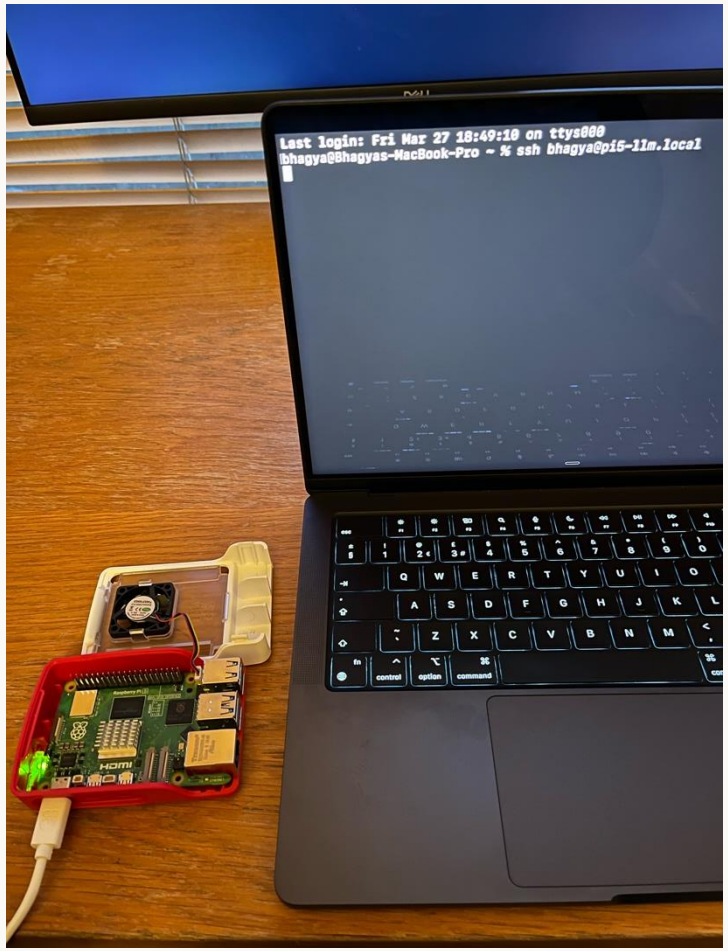
Recommendation

- Recommendation is implemented in bench/recommend.py.
- CSV is filtered to the relevant deployment scenario: device, generation budget and target prompt budget.
- **Hard constraints remove infeasible candidates.**
- These are constraints including maximum RSS, optional memory headroom, maximum TTFT proxy, minimum generation throughput, and, where relevant, minimum context length.
- Then the remaining configurations are ranked based on your specific objective.
- The four ranking objectives that are supported are **speed first**, **quality first**, **balanced**, and **max context.**

UNIVERSITY OF BIRMINGHAM

Portability and robustness

Cross-device validation on *Raspberry Pi 5*



Cross-model validation on *Llama-3.2-1B Instruct*

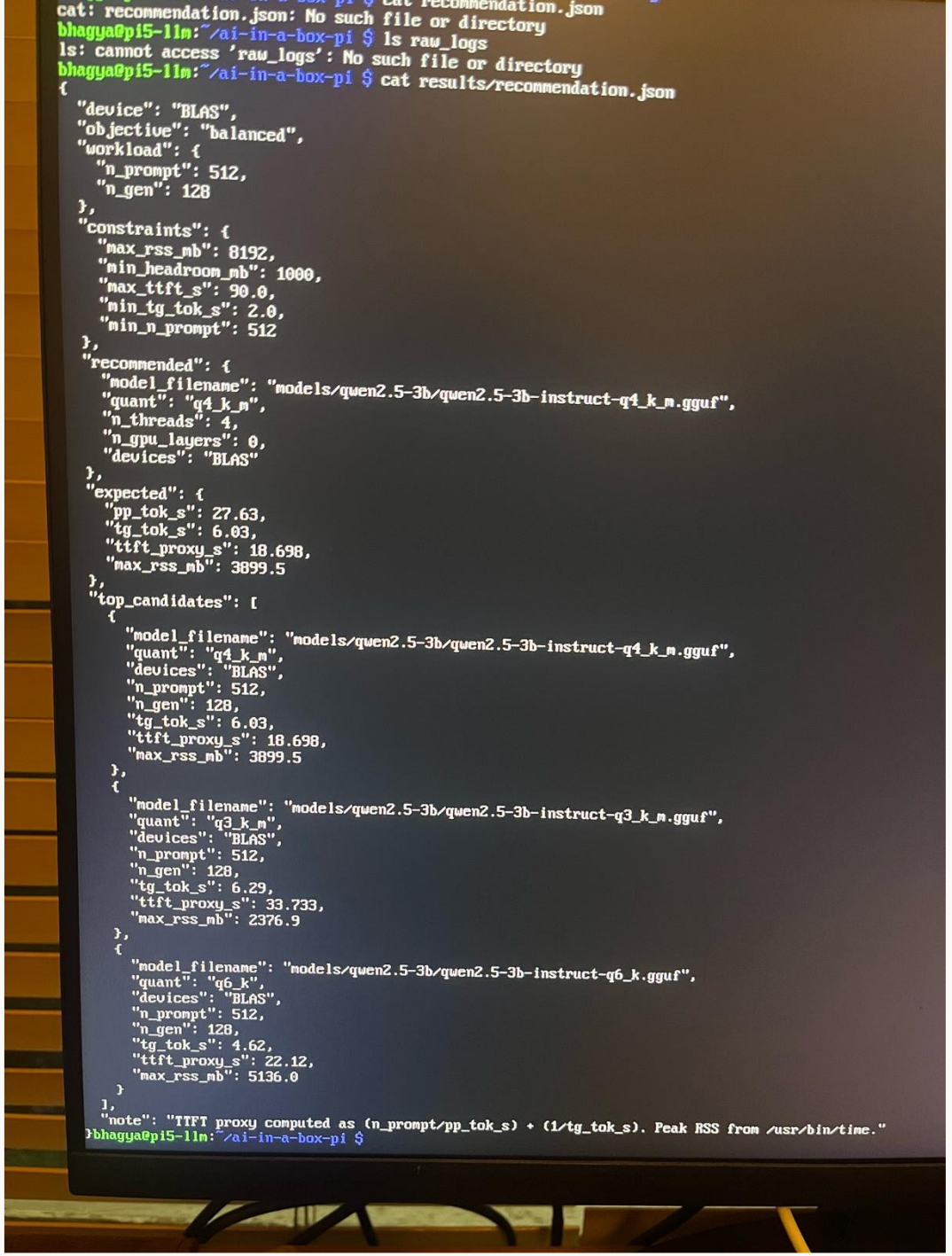
Table 5: Configuration-driven portability sanity test on `Llama-3.2-1B-Instruct-GGUF`. Results are taken from the framework's balanced recommendation query for an `MTL0` `1920/128` workload.

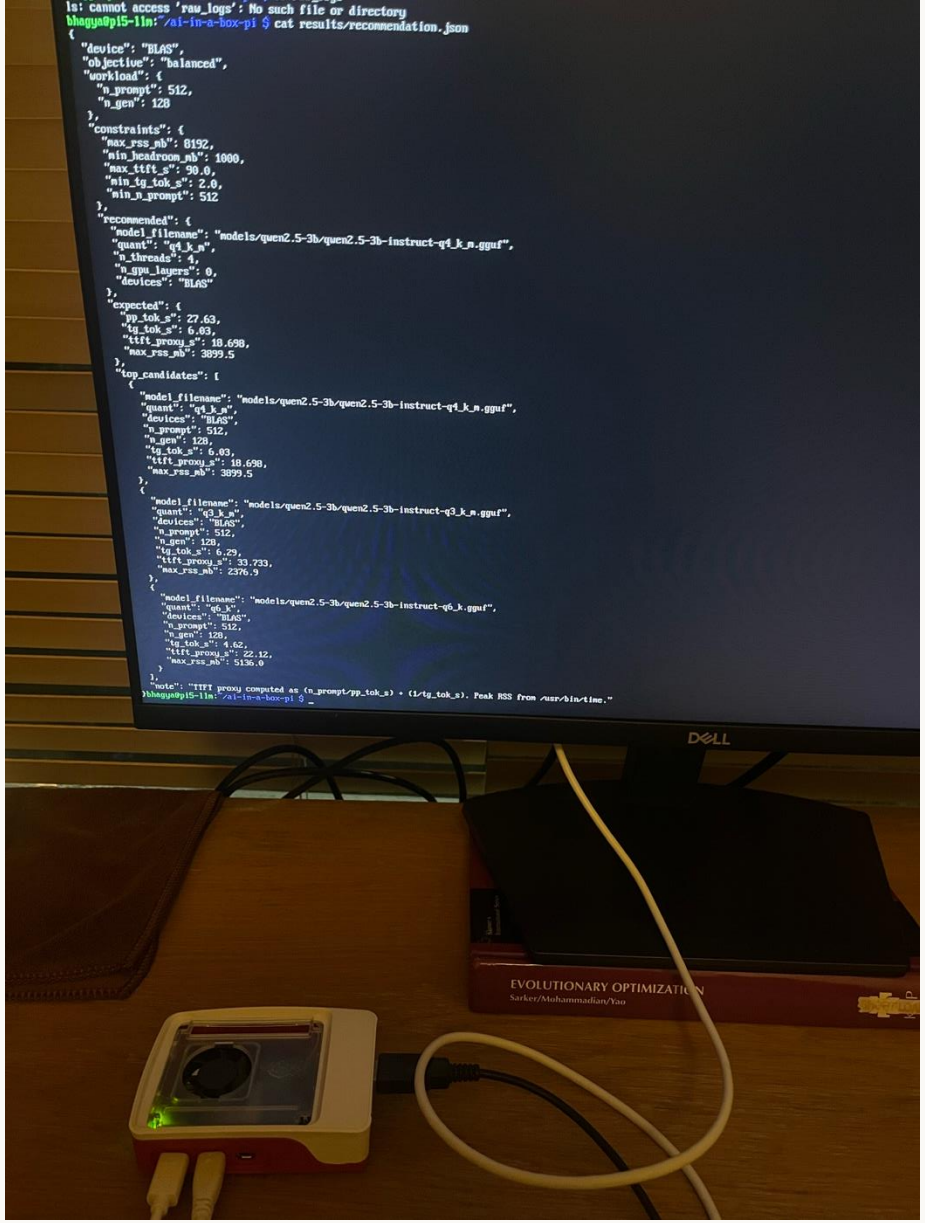
Quant	PP tok/s	TG tok/s	TTFT proxy (s)	RSS (MB)
Q4_K_M	2305.00	197.61	0.838	1360.6
Q6_K	2267.91	166.66	0.854	1565.0
Q8_0	2390.10	150.88	0.807	1971.5

artifacts > llama32_1b_portability_sample > {} recommendation.json > ...


```
{
  "device": "MTL0",
  "objective": "balanced",
  "workload": {
    "n_prompt": 1920,
    "n_gen": 128
  },
  "constraints": {
    "max_rss_mb": 8192,
    "min_headroom_mb": 750,
    "max_ttft_s": 8.0,
    "min_tg_tok_s": 10.0,
    "min_n_prompt": 1920
  },
  "recommended": {
    "model_filename": "models/llama32-1b/Llama-3.2-1B-Instruct-Q4_K_M.gguf",
    "quant": "q4_k_m",
    "n_threads": 8,
    "n_gpu_layers": 80,
    "devices": "MTL0"
  },
  "expected": {
    "pp_tok_s": 2305.0,
    "tg_tok_s": 197.61,
    "ttft_proxy_s": 0.838,
    "max_rss_mb": 1360.6
  },
  "top_candidates": [
    {
```


UNIVERSITY OF BIRMINGHAM





UNIVERSITY OF BIRMINGHAM

Mixed-Precision GGUF Mixing Framework

Part 2



UNIVERSITY OF BIRMINGHAM

arXiv:2306.00978v6 [cs.CL] 25 Apr 2026

AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration

Ji Lin [*1] Jiaming Tang [*12] Haotian Tang [†1] Shang Yang [†1] Wei-Ming Chen [3] Wei-Chen Wang [1]
Guangxuan Xiao [1] Xingyu Dang [14] Chuang Gan [56] Song Han [13]

https://github.com/mit-han-lab/llm-awq

Abstract

Large language models (LLMs) have transformed numerous AI applications. *On-device* LLM is becoming increasingly important: running LLMs locally on edge devices can reduce the cloud computing cost and protect users' privacy. However, the astronomical model size and the limited hardware resource pose significant deployment challenges. We propose Activation-aware Weight Quantization (AWQ), a hardware-friendly approach for LLM low-bit weight-only quantization. AWQ finds that not all weights in an LLM are equally important. Protecting *only 1%* salient weights can greatly reduce quantization error. To identify salient weight channels, we should refer to the activation distribution, not weights. To avoid the hardware-inefficient mix-precision quantization, we mathematically derive that scaling up the salient channels can reduce the quantization error. AWQ employs an equivalent transformation to scale the salient weight channels to protect them. The scale is determined by collecting the activation statistics offline. AWQ does not rely on any backpropagation or reconstruction, so it generalizes to different domains and modalities without overfitting the calibration set. AWQ outperforms existing work on various language modeling and domain-specific benchmarks (coding and math). Thanks to better generalization, it achieves excellent quantization performance for *instruction-tuned* LMs and, for the first time, *multi-modal* LMs. Alongside AWQ, we implement TinyChat, an efficient and flexible inference framework tailored for 4-bit on-device LLM/VLMs. With kernel fusion and platform-aware weight packing, TinyChat offers more than **3**× speedup over the Huggingface FP16 implementation on both desktop and mobile GPUs. It also democratizes the deployment of the 70B Llama-2 model on mobile GPUs.

1 Introduction

Deploying large language models (LLMs) directly on edge devices is crucial. On-device usage eliminates delays caused by sending data to a cloud server and enables LLMs to operate offline, which is beneficial for real-time applications like virtual assistants, chatbots, and autonomous vehicles. The operational costs associated with maintaining and scaling centralized cloud infrastructure can also be reduced.

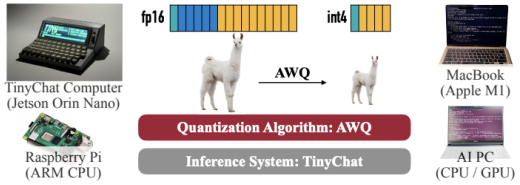


Figure 1. We introduce **AWQ**, a versatile weight quantization method for LLM. To implement AWQ, we developed **TinyChat** to deploy 4-bit quantized LLMs into various edge platforms,

What does *"salient"* mean?

salient

/ˈseɪlɪənt/

adjective

1. most noticeable or important.
   "it succinctly covered all the salient points of the case"
   synonyms: important, main, principal, major, chief, primary, notable, noteworthy, outstanding, arresting, conspicuous, striking, noticeable, obvious, remarkable, signal, prominent, pronounced, predominant, dominant, key, crucial, vital, essential, basic, staple, critical, pivotal, prime, central, focal, paramount
   "the salient points stuck out clearly in her mind"

UNIVERSITY OF BIRMINGHAM

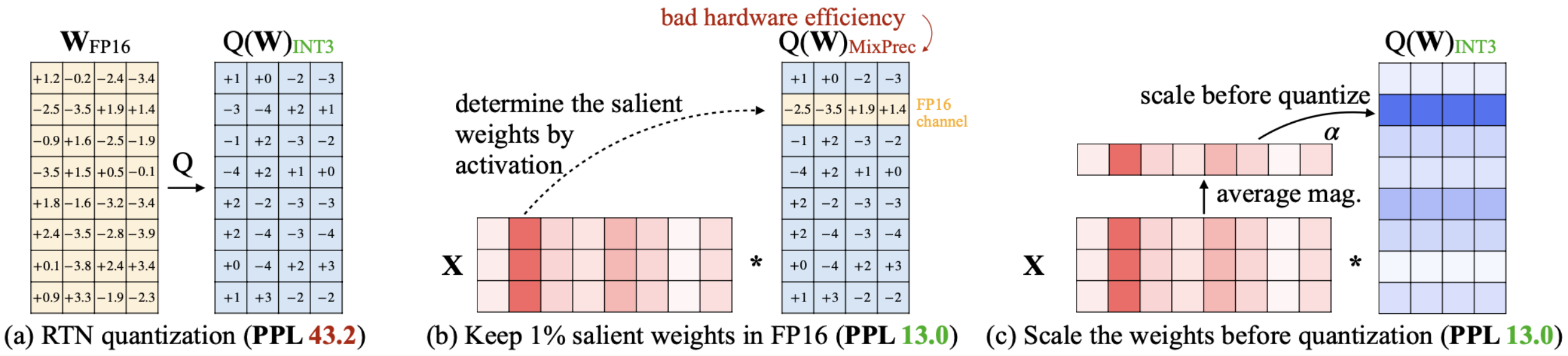
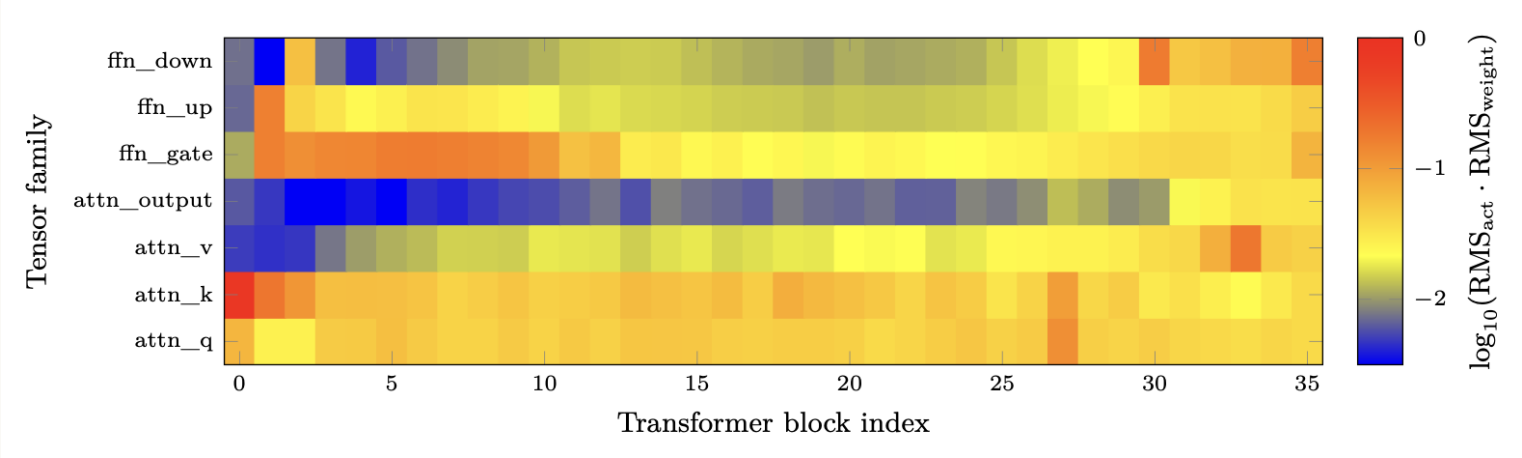


Figure 2. We observe that we can find 1% of the salient weights in LLMs based on the *activation distribution* (middle). Keeping the salient weights in FP16 can significantly improve the quantized performance (PPL from 43.2 (left) to 13.0 (middle)), but the mixed-precision format is not hardware-efficient. We follow the activation-awareness principle and propose AWQ (right). AWQ performs per-channel scaling to protect the salient weights and reduce quantization error. We measure the perplexity of OPT-6.7B under INT3-g128 quantization.

UNIVERSITY OF BIRMINGHAM

Collect Saliency across Tensor Blocks

- Activation-aware saliency collection
- PyTorch forward hook collection
- Hugging Face module name to GGUF tensor mapping
- Saliency JSON output



UNIVERSITY OF BIRMINGHAM

Dynamic Programming Budget Problem

Algorithm 1 Budgeted tensor promotion by forced inclusion, knapsack selection, and greedy fill

Input: low/high GGUF tensor maps, saliency JSON, target extra budget B, forced set F
Output: explicit promotion recipe

Build candidate set C of tensors present in both sources with matching shape, dimension ≥ 2, valid saliency statistics, and positive byte delta
for all $t \in C$ **do**
    $\text{score}(t) \leftarrow \text{RMS}_{\text{activation}}(t) \cdot \text{RMS}_{\text{weight}}(t)$
    $\text{cost}(t) \leftarrow \lceil \Delta\text{bytes}(t)/(0.5\,\text{MiB}) \rceil$
$C_F \leftarrow \{t \in C : t.\text{name} \in F\}, \quad C_R \leftarrow C \setminus C_F$
Subtract the forced byte cost from B; abort if the remainder is negative
Run a 0–1 knapsack over C_R with value $\text{score}(t)$ and bin cost $\text{cost}(t)$
Recover the selected set S from the dynamic-programming table
for all $t \in C_R \setminus S$ in descending order of $\text{score}(t)/\Delta\text{bytes}(t)$ **do**
    if t fits in the remaining byte budget **then**
        add t to S
Output the recipe for $C_F \cup S$ as an explicit tensor-name list

1. Candidate tensor filtering
2. Storage delta computation
3. 0–1 knapsack over 0.5 MiB bins
4. Recipe JSON generation of which tensors to promote

UNIVERSITY OF BIRMINGHAM

GGUF Mixer

- The mix_gguf.py assembles and produces the mixed precision model.
- For each tensor name in the model, the script decides whether to copy from the low precision or high precision source
- Before writing anything, it validates that the two source GGUFs are compatible. Including things like architecture, tensor names and shapes.
- For quantised tensors, the writer must use the tensor's raw packed representation, not its logical matrix shape.
- And then finally based on the recipe budget provided, you set those as high precision and the rest as low precision

UNIVERSITY OF BIRMINGHAM

Results

Table 6: Selected mixed-precision and uniform-quant results under the fixed `MTL0` benchmark configuration (`1920/128`, 8 threads, `-ngl 80`).

Model	Mean PPL	TG tok/s	TTFT proxy (s)	RSS (MB)
Vanilla Q4_K_M	9.069554	83.71	2.288	2425.6
Output + Last 2 Blocks Q8 (≈132 MiB)	9.014643	81.36	2.286	2631.7
Activation-aware, matched budget (≈132 MiB)	9.010936	81.26	2.286	2630.1
Activation-aware, 500 MiB budget	8.903885	73.99	2.261	2995.8
Vanilla Q6_K	8.911915	68.47	2.334	3082.2
Vanilla Q8_0	8.870479	61.98	2.210	3938.7

*Benchmarked based on the Wikitext-2 Dataset

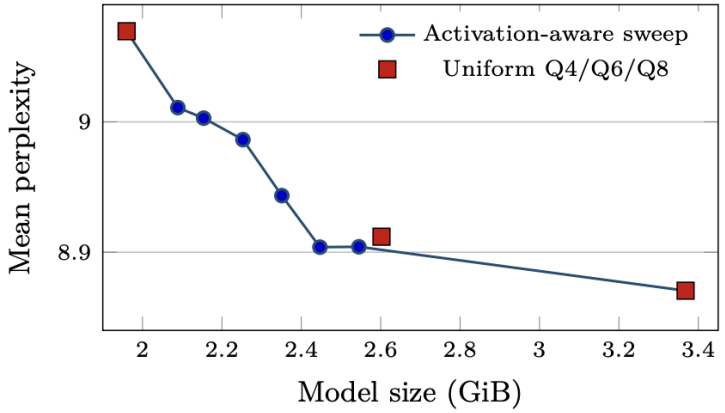
UNIVERSITY OF BIRMINGHAM

Results

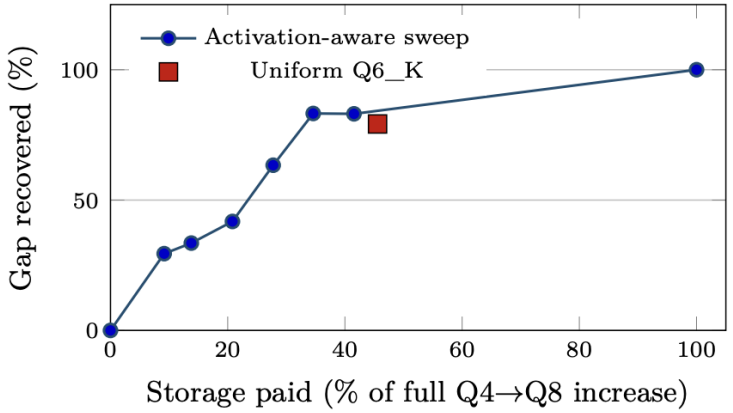
The additional `Q6_K` anchor sharpens that interpretation. Uniform `Q6_K` recovers 79.19% of the `Q4`→`Q8` perplexity gap while paying 45.57% of the storage increase. The 500 MiB mixed model instead recovers 83.22% of the gap while paying only 34.59% of the storage increase. The mixed frontier is therefore not merely interpolating smoothly between `Q4` and `Q8`; at least at this point, it yields a genuinely better quality–storage operating point than the uniform intermediate quant.

Table 7: Activation-aware budget sweep. Storage is reported in GiB from exact file byte counts. "Storage paid" denotes the fraction of the full `Q4_K_M`→`Q8_0` storage increase incurred by each model, while "gap recovered" denotes the fraction of the full `Q4_K_M`→`Q8_0` perplexity gap recovered.

Model	Size (GiB)	Mean PPL	Storage paid (%)	Gap recovered (%)
Vanilla Q4_K_M	1.960	9.069554	0.00	0.00
Act.-aware (≈132 MiB)	2.089	9.010936	9.16	29.45
Act.-aware (200 MiB)	2.154	9.002860	13.79	33.50
Act.-aware (300 MiB)	2.253	8.986345	20.81	41.80
Act.-aware (400 MiB)	2.351	8.943408	27.76	63.37
Act.-aware (500 MiB)	2.447	8.903885	34.59	83.22
Act.-aware (600 MiB)	2.545	8.904166	41.55	83.08
Vanilla Q8_0	3.368	8.870479	100.00	100.00



(a) Quality as a function of size, with `Q6_K` as a uniform-quant anchor.



(b) Gap recovered versus storage paid, showing the non-linear frontier.

Figure 6: Mixed-precision quality–storage trade-offs. The frontier improves rapidly at modest budgets, the 500 MiB mixed model slightly outperforms uniform `Q6_K` while using less storage, and gains flatten beyond that point.

UNIVERSITY OF BIRMINGHAM

Limitations and Conclusion

- Limitations:
  - The majority of the framework is model and device agnostic apart from the part 2 saliency mapping which is tied closely to Qwen 2.5 Instruct
- Contributions:
  - Portable benchmarking and recommendation framework/toolkit
  - Mixed-precision GGUF mixing framework
- Conclusion:
  - Deployment should be empirical and data driven
  - Selective promotion can disproportionately recover quality with selective quantisation promotion.